Master Thesis

Entwicklung eines interaktiven HMI-Konzepts mit Smart Knob für moderne Fahrzeugbedienung

Interactive Human-Machine Interface Design for Automotive Use: Integration of Smart Knob and Visual Feedback Elements

Shubham Tandon

January 2026

Abstract

This master's thesis presents the development of an interactive Human-Machine Interface (HMI) concept for modern vehicle operation, centred around a haptic smart knob integrated with a compact round display and an ESP32 microcontroller platform. The system functions as a driver-facing control and information hub, demonstrating how safety-critical driving variables — including vehicle speed, virtual torque output, and engine RPM — can be communicated in a perceptually efficient manner while remaining operable through a single multifunctional rotary control element.

A central concern motivating this work is the growing tension between the ever-increasing functional richness of modern in-vehicle interfaces and the attentional constraints imposed by the primary driving task. Current touchscreen-dominated HMI architectures demand sustained visual engagement, thereby elevating the risk of driver distraction. The proposed smart-knob approach addresses this tension by consolidating mode selection, state confirmation, and real-time information display into one co-located interaction unit, reducing the need for extended gaze shifts to a central display.

The practical contribution of this thesis encompasses three areas. First, the mechanical and electronic integration of the smart knob assembly into a functional demonstrator, including a brushless DC haptic actuator, a magnetic absolute encoder (MT6701), an HX711-based strain gauge for press detection, and an eight-LED NeoPixel ring for peripheral visual feedback. Second, the development of embedded firmware implementing a closed-loop haptic detent control system with three stable rotational positions, each mapped to a distinct instrument UI view rendered on a 1.28 in round GC9A01 TFT display. Third, the design of a serial communication protocol enabling synchronisation with a laptop-based HTML dashboard, which acts as the signal master for simulated vehicle data.

Evaluation of the prototype addresses operational effort, response latency, and subjective user acceptance. The results are discussed in the context of established automotive HMI design principles, and recommendations are derived for the integration of combined haptic-visual control concepts in future vehicle cockpit architectures.

Contents

Abstract **i**

1 Introduction **1**
1.1 Motivation and Context . . . 1
1.2 Research Objectives and Research Questions . . . 2
1.3 Scope and Delimitations . . . 2

2 Problem Description **3**
2.1 Complexity Explosion and Functional Overload . . . 3
2.2 Loss of Haptic Feedback and the Targeting Problem . . . 4
2.3 Distraction, Workload, and Road Safety . . . 4
2.4 Trust, Automation, and Supervisory Control . . . 4
2.5 Summary . . . 5

3 Concept for a Smart-Knob-based HMI **6**
3.1 Local Visual Feedback on the Round Display (GC9A01) . . . 6
3.1.1 UI Views and Information Prioritisation . . . 6
3.1.2 Rendering Strategy and Display Stability . . . 8
3.1.3 Local Display as a Cognitive Offloading Tool . . . 8
3.2 Haptic Feedback Concept . . . 8
3.2.1 Detent-Based Mode Selection . . . 8
3.2.2 Resistance and Detent Feel: the Haptic Control Law . . . 9
3.2.3 Haptic Press Confirmation via Strain Gauge . . . 10
3.3 Optional Voice Interaction Concept (Future Extension) . . . 10
3.4 Ergonomic Placement and Cockpit Integration Considerations . . . 11
3.4.1 Interaction Loop Summary . . . 11

4 System Architecture and Hardware Design **12**
4.1 System Overview . . . 12
4.2 Hardware Components . . . 12
4.2.1 ESP32 Microcontroller Platform . . . 12
4.2.2 1.28-inch Round GC9A01 Display Module . . . 13
4.2.3 Rotary Encoder, BLDC Motor, and Haptic Driver . . . 13
4.2.4 Strain Gauge and HX711 ADC . . . 14
4.2.5 NeoPixel LED Ring . . . 14
4.3 Electronics Integration and Power Supply . . . 14
4.4 Mechanical Design and Mounting . . . 15
4.5 Reference Project: SmartKnob . . . 15
4.6 Interfaces to Simulated Vehicle Signals . . . 15

5 Software and Interaction Logic **17**
5.1 Firmware Architecture Overview . . . 17
5.2 Included Libraries and Core Objects . . . 17
5.3 UI Mode Management . . . 18
5.4 Haptic Control Task: Encoder, Control Law, and BLDC Commutation . . . 18
5.5 Strain Gauge Sampling and LED Feedback . . . 19
5.6 Speedometer UI Rendering . . . 20
5.7 Initialisation and Main Loop . . . 21
5.8 Laptop Dashboard and Synchronisation . . . 22
5.9 Sensor and Peripheral Communication Protocols . . . 22
5.9.1 GC9A01 TFT Display — Hardware SPI (Unidirectional) . . . 23
5.9.2 TFT Backlight — LEDC PWM (Unidirectional) . . . 23
5.9.3 MT6701 Magnetic Encoder — SSI Protocol (Unidirectional, Sensor to ESP32) . . . 23
5.9.4 HX711 Strain Gauge ADC — Custom Serial Protocol (Unidirectional, Sensor to ESP32) . . . 24
5.9.5 NeoPixel LED Ring — WS2812B Single-Wire Protocol (Unidirectional) . 24
5.9.6 BLDC Motor via TMC6300 — LEDC PWM (Unidirectional) . . . 25
5.9.7 Laptop Serial Link — UART over USB (Bidirectional) . . . 25

6 Experimental Setup and Evaluation **26**
6.1 Evaluation Goals and Hypotheses . . . 26
6.2 Participants . . . 27
6.3 Experimental Task Design . . . 27
6.4 Apparatus and Procedure . . . 27
6.5 Questionnaire Instrument . . . 28
6.6 Scoring and Analysis Plan . . . 30
6.7 Results . . . 30
6.8 Discussion . . . 30

7 Conclusion and Outlook **31**
7.1 Summary of Contributions . . . 31
7.2 Implications for Automotive HMI Design . . . 31
7.3 Recommendations for Future Work . . . 32
7.4 Potential Integration into Production Vehicle Cockpits . . . 32

A Hardware Datasheets and Component References **33**
A.1 Principal Components . . . 33

B Additional Schematics and Source Code **34**
B.1 User_Setup.h Configuration . . . 34

C Questionnaire and Evaluation Material **35**
C.1 Post-Session Questionnaire . . . 35

D Full Firmware Source Code **36**

Chapter 1

Introduction

1.1 Motivation and Context

The cockpit of a modern automobile has undergone a profound transformation over the past two decades. Where earlier generations of vehicles equipped their dashboards with purpose-built physical controls — dedicated knobs for climate, individual switches for lighting, analogue instruments for speed and engine load — contemporary vehicles increasingly consolidate these functions behind large touchscreen surfaces and hierarchically organised software menus. This architectural shift has been driven by a convergence of factors: the desire to reduce manufacturing cost and panel complexity, the expectation of consumers who are accustomed to smartphone interaction paradigms, and the commercial pressure to differentiate vehicles through continuously updatable digital features.

At the same time, the number of functions accessible to the driver has grown dramatically. Navigation, over-the-air software updates, streaming media, advanced driver assistance configuration, personalised driving mode selection, and real-time vehicle telemetry are now standard offerings in the mid-range and premium segments. The natural consequence is a significant increase in the cognitive and visual workload imposed on the driver — precisely at a time when the regulatory and societal expectation is that drivers should remain attentive, responsive, and in control of the vehicle at all times.

This thesis is motivated by the observation that the dominant touchscreen-centric paradigm, while powerful and flexible, is not inherently well-suited to the context of driving. A flat glass surface offers no spatial or tactile cues: every interaction requires the driver to shift their gaze away from the road, visually locate the correct on-screen element, and confirm through visual inspection that the intended action was registered. This sequence creates what researchers in the field of driver behaviour refer to as a *secondary task burden*: a measurable increase in eyes-off-road time and a reduction in the attentional resources available for hazard detection and vehicle control.

Against this background, the *smart knob* emerges as a compelling design alternative. A smart knob is a single rotary control element that combines physical interaction — rotation, detented positioning, and press detection — with software-configurable haptic feedback and closely coupled local visual output. Because it is physically grounded, it can in principle be located and operated by touch alone, without requiring the driver to look at the control. Because it provides haptic confirmation (a perceivable "click" as a detent is reached), it communicates state changes through the driver's tactile channel rather than the visual channel. And because a compact display is integrated into or immediately adjacent to the knob, the information relevant to the current control context can be presented locally, without requiring a gaze shift to a

remote central screen.

This work develops a full prototype implementation of such a concept, grounded in the specific hardware and firmware constraints of an ESP32-based embedded system, and evaluates it as an HMI candidate for automotive instrument and control applications.

1.2 Research Objectives and Research Questions

The overarching objective of this thesis is to demonstrate the feasibility of a smart-knob-based HMI concept that reduces visual-manual demand in a multi-function vehicle control scenario, and to evaluate the extent to which such a concept fulfils core automotive HMI usability criteria.

This objective is decomposed into the following specific research questions:

RQ1. To what extent can a single rotary haptic control element provide reliable, eyes-free mode selection across multiple instrument views in an automotive context?

RQ2. How can visual feedback on a compact round display be structured to support fast, glanceable information uptake for safety-relevant driving variables (speed, torque, RPM)?

RQ3. What are the critical firmware design constraints for a real-time closed-loop haptic detent system running concurrently with display rendering on a single ESP32 platform?

The contributions of this thesis are: (i) a complete working prototype integrating haptic actuation, magnetic angle sensing, strain-based press detection, NeoPixel LED feedback, and multi-mode TFT visualisation; (ii) a documented firmware architecture addressing the real-time and concurrency requirements of closed-loop haptics and display rendering; and (iii) an initial evaluation providing empirical evidence regarding interaction quality, distraction reduction, and user acceptance.

1.3 Scope and Delimitations

This work covers the design, implementation, and initial laboratory evaluation of a smart-knob HMI prototype. The system processes simulated vehicle signals (speed, torque, RPM) injected via a serial link from a laptop; it does not interface directly with a vehicle bus (CAN, LIN, or Ethernet) in the current prototype stage. The evaluation is conducted in a controlled bench-test environment; on-road or driving-simulator studies are identified as future work.

The haptic actuation is implemented using a consumer-grade brushless gimbal motor and a 10-bit PWM drive architecture; the torque output is therefore limited in magnitude and precision compared to industrial-grade haptic actuators. The visual display is a 1.28 in round TFT and is not intended to replace a primary instrument cluster; rather, it functions as a supplementary, co-located feedback channel. Voice interaction is described at the concept level but is not implemented in the embedded firmware in the current version.

Chapter 2

Problem Description

The core problem of today's automotive Human-Machine Interfaces (HMI) lies in a simple but fundamental tension. On the one hand, cars are increasingly marketed and used as multifunctional spaces for communication, work, and entertainment. On the other hand, the basic task of driving has not changed: it still requires continuous attention and leaves only limited spare capacity for secondary activities. As HMI systems become richer and more complex, they risk overloading the driver, increasing cognitive effort, and creating situations in which important events in the traffic scene are missed or reacted to too slowly.

The challenges of modern in-vehicle HMI can be described along four closely related dimensions.

2.1 Complexity Explosion and Functional Overload

Older vehicles offered only a small set of functions, and most of them followed a clear one-to-one mapping between a physical control and a specific action. By contrast, current premium vehicles expose several hundred distinct functions to the user, spanning powertrain modes, driver assistance settings, comfort features, infotainment options, and connectivity services. Industry analyses suggest that a high-end luxury sedan can provide on the order of several hundred, and in some cases approaching 700, individually accessible settings and parameters.

It is neither practical nor ergonomically desirable to implement hundreds of dedicated hardware buttons. As a result, most functions are now accessed through hierarchical menus presented on one or more central touchscreen displays. Functions are grouped into layers and submenus, much as they are in desktop operating systems or smartphone applications. The consequence is that even very simple adjustments — for example, modifying a climate zone setting, selecting a media source, or adjusting an assistance parameter — may require several discrete interaction steps: locating the correct menu category, visually scanning a list, selecting the appropriate entry, and confirming the change.

This structural complexity has two important consequences for the driver. First, visual and cognitive resources must be invested in menu navigation while the vehicle is in motion; these are precisely the same resources required for hazard perception and vehicle control. Second, the overall goal structure of a typical journey has broadened: many drivers expect to manage navigation, communication, and entertainment concurrently with the primary task. The more complex and visually engaging the in-vehicle interface becomes, the more intensely it competes for the same limited attentional capacity that safe driving demands.

2.2 Loss of Haptic Feedback and the Targeting Problem

Alongside this functional expansion, the physical character of in-vehicle controls has changed fundamentally. Traditional buttons, rotary knobs, and toggle switches have been progressively replaced by large flat touchscreen panels. This transition removes an important safety characteristic of earlier control architectures: the tactile and kinaesthetic structure that allowed drivers to locate and confirm control actions without sustained visual attention.

With mechanical controls, drivers can often identify and operate a function largely by touch. The shape, texture, and spatial position of a knob or switch provide passive orientation cues, and the physical state of the control — a distinct detent position, a firm end stop, a toggle's binary snap — can frequently be sensed without any direct visual engagement. A smooth glass surface offers virtually none of this passive structure. To activate a virtual button or move a slider on a touchscreen, the driver must typically perform the following sequence:

1. Direct gaze at the display to locate the desired control element.
2. Move the hand towards the target while continuously using visual feedback to correct the approach trajectory.
3. Verify on the screen that the correct element was activated and that the intended change was registered.

This sequence constitutes what may be termed a *targeting requirement*: reliable interaction with on-screen controls demands repeated, visually guided correction. Interaction patterns that are natural and low-effort on consumer handheld devices — scrolling through lists, manipulating sliders, tapping small icons — are therefore inherently demanding in a driving context, where the forward roadway simultaneously requires continuous monitoring. The net result is a shift in the characteristic gaze pattern: from a regime of primarily forward-facing attention with brief confirmatory glances at controls, towards extended and repeated episodes of eyes-off-road engagement with an in-vehicle screen.

2.3 Distraction, Workload, and Road Safety

The effects of visually demanding in-vehicle HMIs are reflected in broader accident and risk statistics, even where it is not possible to isolate the contribution of a specific system in individual incidents. Naturalistic driving studies and simulator research consistently report elevated secondary-task reaction times, increased lane deviation variability, and reduced hazard detection rates during touchscreen interaction compared to equivalent physical-control tasks. Regulatory bodies in several jurisdictions have begun to quantify acceptable eyes-off-road times and interaction durations, acknowledging that the current generation of touchscreen HMIs frequently exceeds these limits for common operations. These findings provide the empirical foundation for seeking alternative interaction architectures that can maintain functional richness while reducing the visual and manual demand placed on the driver.

2.4 Trust, Automation, and Supervisory Control

Modern vehicles increasingly incorporate Advanced Driver Assistance Systems (ADAS) and partially automated driving functions. This development shifts a portion of the HMI challenge

from the direct execution of control actions towards the supervision of semi-autonomous systems and the management of appropriate trust levels. Two dysfunctional patterns are recurrently described in the human factors literature:

- **Misuse**: When the interface fails to communicate the operational limits or boundary conditions of an assistance system clearly and unambiguously, drivers may form inflated expectations of system capability, leading to over-reliance and inattention during periods when the system is operating at or near its limits.
- **Disuse**: When the interface is perceived as overly complex, inconsistent, or intrusive, drivers may actively disable or avoid assistance functions that could otherwise contribute to safety, sacrificing the benefit of the technology due to poor usability.

2.5 Summary

In summary, the current state of automotive HMI can be characterised as a safety-usability trade-off operating under conditions of rapidly increasing functional complexity. Modern digital interfaces deliver convenience, connectivity, and personalisation, but impose elevated visual, manual, and cognitive demands. The transition from dedicated, tactile controls to large menu-driven touchscreens has made it more difficult for drivers to execute routine operations by touch alone, and has increased reliance on visually guided, multi-step interaction sequences.

This problem description motivates the central question pursued in this thesis: how can alternative HMI concepts — specifically, a smart-knob-based architecture combining local haptic feedback with a tightly coupled visual display — be designed such that they preserve access to the necessary vehicle functions while substantially reducing visual-manual demand and cognitive load, and thereby better support the primary task of safe vehicle operation?

Chapter 3

Concept for a Smart-Knob-based HMI

This chapter derives a concrete interaction concept for a smart-knob-based HMI and maps it to an implementable prototype architecture. The concept addresses the core HMI problem described in Chapter 2: modern vehicle interfaces risk functional overload and high visual-manual demand, while the driving task requires robust, glanceable information access and low cognitive overhead. The smart knob aims to resolve this tension by consolidating *mode selection*, *action confirmation*, and *state feedback* into a single primary control element. The user switches between instrument views through a physically structured rotary action and receives immediate confirmation through haptic detents and co-located visual feedback. In the present prototype, the knob integrates (i) a round display for glanceable instrument-style feedback, (ii) a haptic motor control loop generating configurable detents and resistance profiles, (iii) a strain-gauge-based press detection mechanism, and (iv) a NeoPixel LED ring providing peripheral state indication.

3.1 Local Visual Feedback on the Round Display (GC9A01)

A primary design goal is to provide *glanceable* information that can be interpreted with minimal reading effort and stable rendering under varying viewing angles. To this end, the prototype employs a 1.28 in round TFT display (GC9A01 controller, 240×240 px) to present compact, instrument-style views. The chosen visual language deliberately adopts established automotive instrument patterns — circular dials, scale tick marks, colour-coded zones, a pointer needle, and a prominent numerical readout — enabling rapid recognition through the driver's pre-existing familiarity with this visual vocabulary.

3.1.1 UI Views and Information Prioritisation

The smart knob provides three distinct UI modes, each rendered on the GC9A01 display using the `TFT_eSPI` graphics library, a high-performance SPI-based TFT driver well suited to ESP32-class microcontrollers. All three views share a consistent visual logic: a near-black dial interior maximises contrast, suppresses glare on the small screen surface, and ensures that the primary numeric readout remains legible at a brief glance. The three views are as follows.

Speedometer view (default; safety-critical). The speedometer view is designated as the default power-on mode, reflecting the primacy of speed information for safe driving. The implementation combines a circular dial with graduated tick marks, a filled-triangle pointer needle, and a central numeric speed readout, supplemented by a gear indicator displaying

current gear or a neutral state (`N`). To support rapid interpretation, the dial arc is partitioned into three colour-coded zones rendered via interpolated triangular primitives: green for the low-speed range (0–80 km/h), transitioning through yellow (80 —-140 km/h), to red for the high-speed range (140 —-200 km/h). This colour mapping is intentionally redundant: the driver can assess the current speed state either through the precise numeric value or through the needle's position within the colour zone, without requiring exact scale reading. Speed labels are rendered at 40 km/h intervals, with major and minor tick marks at 20 km/h and 10 km/h increments, respectively. The gear indicator is displayed in a rounded rectangle at the lower portion of the dial in a contrasting amber tone (`TFT_ORANGE`), ensuring rapid gear-state discrimination.

Automatic gear inference is implemented in the firmware: gear assignments are computed from the reported speed value according to a fixed mapping (neutral at 0, gears 1 through 6 at increasing speed bands), providing a plausible simulated powertrain state for demonstrator purposes without requiring explicit gear input.

Torque view (performance feedback and short-term dynamics). The torque view presents the instantaneous virtual torque value in Newton-metres through three complementary visual channels: a colour-interpolated progress arc, a directional pointer needle, and a centred numeric readout. The arc fill conveys coarse magnitude information ("how much torque relative to the configured maximum"); the needle enables more precise perception of the current position within the scale; and the numeric value provides unambiguous quantitative reference. Additionally, a *peak torque* indicator is maintained in the firmware and displayed below the central readout. This peak indicator preserves a short-term memory of the maximum recent torque demand, which is particularly informative during dynamic operation (simulated acceleration events, rapid throttle input, or acoustic-coupled EV demonstrator scenarios) where the instantaneous value may be highly transient.

The configurable maximum torque parameter (`TORQUE_MAX_NM`, default 50 Nm, adjustable via serial command `M`) allows the scale to be adapted to different simulated drivetrain characteristics without firmware recompilation, increasing the demonstrator's flexibility. The needle colour transitions from green to yellow to red as the torque fraction exceeds 40% and 75% of the configured maximum, respectively, providing an additional visual urgency cue consistent with the speedometer's colour coding convention.

RPM view (performance impression and EV analogue feedback). The RPM view reproduces a classic tachometer aesthetic to provide a sport-oriented performance feedback channel. It incorporates a circular scale from 0 to 8000 RPM, graduated tick marks at 500 RPM intervals with labelled major ticks at every 1000 RPM, a filled-triangle needle, and a central numeric RPM readout. Even within an electric vehicle demonstrator context, RPM provides value as a *perceptual proxy variable*: it allows otherwise silent electric acceleration to be represented in a familiar, engine-like performance register, particularly when coupled with synthetic sound generation or haptic feedback intensity modulation. The needle follows the same three-state colour scheme (green below 3000 RPM, yellow between 3000 and 6000 RPM, red above 6000 RPM).

Mode access and prioritisation logic. The three views are arranged in an explicit priority hierarchy: the speedometer is the default and safety-primary view; torque and RPM are secondary views accessible on demand. This hierarchy is enforced both by the power-on default state and by the direction of mode cycling. In terms of automotive HMI design principle, the

arrangement reflects the maxim that *mandatory driving information should be primary; optional performance feedback should be available but non-dominant.*

3.1.2 Rendering Strategy and Display Stability

To avoid perceptible flicker and maintain visual stability on the high-contrast round display, the firmware differentiates between two classes of graphical elements:

- **Static base elements**: dial background, concentric border rings, tick marks, scale labels, colour zones, and unit indicators. These are drawn once when a mode is entered (`drawUI()`, `torque_drawBase()`, `rpm_drawBase()`) and are not redrawn on subsequent update cycles.
- **Dynamic elements**: the pointer needle and the central numeric readout. These are updated on every main-loop iteration in which the underlying value has changed, at a period of 50 ms (`LOOP_PERIOD`).

Needle updates are implemented by first overdrawing the previous needle position in the background colour (black), then drawing the needle at the new position in the appropriate colour. This partial-redraw strategy avoids full-screen refresh overhead and yields a stable, responsive display with no perceptible flicker on the 240×240 round panel. The approach directly addresses a recognised usability issue in embedded display applications: visually noisy updates extend the required glance duration, increasing driver distraction.

3.1.3 Local Display as a Cognitive Offloading Tool

The co-location of the display with the rotary control element is a deliberate architectural decision. In a traditional cockpit layout, mode changes or parameter adjustments made at a central console require the driver to shift gaze to a remote primary display to confirm the effect of the action. By embedding a display directly within the control unit, the smart knob enables the driver to perceive the result of a rotary action within the same spatial area where the action was performed, minimising gaze travel distance. The display therefore functions not as an infotainment screen but as a *tight-loop context indicator* that closes the action-feedback loop locally.

3.2 Haptic Feedback Concept

Haptic feedback is employed to provide *eyes-free* mode-change confirmation and to enforce a stable, predictable mode-selection behaviour. This is particularly important in the automotive context, where visual confirmation requires a gaze shift; a well-designed haptic signal communicates state transitions through the tactile channel with essentially zero visual demand.

3.2.1 Detent-Based Mode Selection

The primary rotational interaction is structured around a detent system with three stable equilibrium positions, spaced 120° apart in mechanical angle. Each detent corresponds to exactly one UI view: detent 0 to the speedometer, detent 1 to the torque display, and detent 2 to the RPM tachometer. The user rotates the knob until the next detent is reached; at that point, the display mode transitions and the driver feels a perceptible resistive snap-in confirming the selection.

From a control architecture perspective, the detent system is implemented as a real-time feedback loop executing as a dedicated FreeRTOS task pinned to Core 0 of the ESP32, separate from the display and serial-handling logic on Core 1. The loop performs the following operations on each iteration:

- **Angle acquisition**: the MT6701 absolute magnetic encoder is read via a software-emulated SSI protocol, providing a 14-bit angular measurement with $360° / 2^{14} = 0.022°$ resolution.
- **Nearest-detent assignment**: the calibrated angle is mapped to the nearest of three detent centres (0°, 120°, 240°).
- **Hysteresis**: a hysteresis band of 25° prevents oscillation between adjacent detent assignments at the midpoint.
- **Torque computation**: a proportional-derivative control law computes the target motor torque.
- **BLDC commutation**: the computed torque is converted to three-phase duty cycles and applied to the motor driver.
- **Stability gating**: a mode-change event is published to the main loop only after five consecutive consistent detent classifications (`STABLE_THRESH` = 5), preventing transient noise from triggering unintended transitions.

3.2.2 Resistance and Detent Feel: the Haptic Control Law

The perceived detent is generated by actively commanding a motor torque that opposes angular deviation from the nearest detent centre, analogous to a virtual torsional spring. The implemented control law is:

$$\tau = K_P \cdot e(\theta) - K_D \cdot \dot{\theta} + \tau_{\text{stick}}(e), \tag{3.1}$$

where θ denotes the calibrated mechanical angle of the shaft, $\dot{\theta}$ is the estimated angular velocity (computed from consecutive encoder readings and elapsed time), and $e(\theta)$ is the signed angular error to the current target detent centre. The parameter $K_P = 1.5$ defines detent stiffness (the "spring" strength), $K_D = 0.004$ provides velocity damping (suppressing overshoot and mechanical resonance), and τ_{stick} is a static friction term applied when the error is below 12°:

$$\tau_{\text{stick}}(e) = \mu_s \cdot \left(1 - \frac{|e|}{12}\right) \cdot \text{sgn}(e), \quad \mu_s = 0.20, \tag{3.2}$$

which creates a progressive snap-in effect as the knob approaches the detent centre. The total torque command is clamped to $\pm$`TORQUE_LIMIT` = 0.80 (normalised to the motor driver's full-scale duty cycle). A three-phase sinusoidal commutation strategy (`driveBLDC()`) translates the scalar torque command into phase PWM duty cycles, with the electrical angle computed from the mechanical angle and the motor's pole pair count ($N_p = 7$):

$$\theta_{\text{elec}} = N_p \cdot \theta_{\text{mech}} + 90°. \tag{3.3}$$

The phase voltages are set as:

$$d_U = 0.5 + 0.5 \cdot |\tau| \cdot \sin(\theta_{\text{elec}} + \phi), \tag{3.4}$$
$$d_V = 0.5 + 0.5 \cdot |\tau| \cdot \sin(\theta_{\text{elec}} + \phi - 120^\circ), \tag{3.5}$$
$$d_W = 0.5 + 0.5 \cdot |\tau| \cdot \sin(\theta_{\text{elec}} + \phi + 120^\circ), \tag{3.6}$$

where $\phi = +90^\circ$ for positive torque and -90° for negative torque, ensuring the force vector is always orthogonal to the rotor flux and therefore maximally effective. A hardware constraint of the prototype board requires that pin IO21 — which is shared with the TFT chip-select line (`TFT_CS`) — is excluded from motor phase driving; accordingly, only the five remaining half-bridge channels (UH, UL, VH, VL, WH) are used, and the WL phase is held inactive.

3.2.3 Haptic Press Confirmation via Strain Gauge

Beyond rotational detents, the prototype implements a press-like interaction modality using a load cell (HX711 24-bit ADC with a strain bridge). The raw strain signal is averaged over an eight-sample sliding window to suppress quantisation noise and mechanical vibration. The filtered value is compared to a zero-reference measured at startup (`calibrateStrainGauge()`, average of 20 readings) and the absolute deviation is compared against an adjustable threshold (`strain_threshold`, default 70000 counts). When the threshold is exceeded, the interaction is interpreted as a deliberate press event and two feedback channels are activated simultaneously:

1. **Visual**: the NeoPixel LED ring transitions from its idle dim-green state to full-brightness red, providing an immediately visible peripheral confirmation.

2. **Haptic**: a short vibration pulse is generated by alternating the applied motor torque direction in rapid succession (eight half-cycles of ± 0.65 normalised torque at 10 ms intervals), producing a brief tactile burst analogous to the click of a mechanical button.

Press event detection employs a rising-edge gate (`ledWasRed` state variable), ensuring that the vibration pulse fires exactly once per press event rather than continuously during sustained contact. This edge-detection mechanism is essential for correct multimodal feedback: a prolonged vibration would be perceived as more intrusive than a brief confirmation pulse and would interfere with the rotational haptic profile.

3.3 Optional Voice Interaction Concept (Future Extension)

As a forward-looking extension of the multimodal interaction concept, voice control is identified as a complementary input channel for situations in which the driver's hands should remain on the steering wheel. In the current system architecture, voice command processing is implemented at the laptop side as a prototype software layer, with recognised commands transmitted to the ESP32 over the existing serial link. The command vocabulary is intentionally minimal to reduce recognition ambiguity: mode-switching commands (`SPEED`, `TORQUE`, `RPM`, `CHANGE`) and flow-control commands (`stop`, `resume`) are sufficient to replicate all primary knob interactions. Each recognised voice command produces identical state transitions to the corresponding physical knob action, ensuring consistent system behaviour across input modalities. Voice interaction is treated as a non-critical redundancy channel: the system remains fully functional without it, and its primary role in the present context is as a demonstrator component for evaluating multimodal interaction preference and distraction.

3.4 Ergonomic Placement and Cockpit Integration Considerations

The ergonomic and spatial placement of the smart knob within the vehicle cockpit significantly influences its practical safety impact. The concept assumes placement within comfortable reach of the driver's primary hand position — for example, on the centre console adjacent to the gear selector or on a steering-wheel spoke insert — such that operation requires minimal arm extension or postural shift. Key ergonomic requirements include:

- **Reach envelope**: the device should be operable from the natural driving posture without requiring the driver to lean laterally or extend the arm beyond a comfortable range.
- **Glance legibility**: the display should be legible within a single short glance (1 –-2 s), achieved through large primary fonts, high contrast, and a stable rendering strategy (no background flicker).
- **Tactile guidance**: the detent profile should be strong enough for reliable mode identification by feel alone, without requiring visual confirmation of knob position.
- **Consistency and learnability**: the fixed mapping of three detent positions to three instrument views must remain invariant across all operating conditions, enabling robust muscle-memory formation.

3.4.1 Interaction Loop Summary

The complete user interaction loop supported by the prototype can be summarised as follows:

1. The driver rotates the knob; the haptic control law guides the knob towards the nearest detent and provides a perceptible resistive snap at the detent centre.
2. Upon stable detent arrival (after five consistent classifications), the display transitions to the corresponding instrument view (speedometer, torque gauge, or tachometer), and a mode-change event is reported over the serial link.
3. The driver presses the knob (strain threshold exceeded); the LED ring changes from dim green to full red, a haptic vibration pulse confirms the press event, and the event is logged over the serial link.

This loop provides a compact, multimodal solution to the automotive HMI problem identified in Chapter 2: multi-function access is achieved with substantially reduced visual demand by combining locally rendered display feedback with tactile state confirmation.

Chapter 4

System Architecture and Hardware Design

4.1 System Overview

The prototype system consists of three principal subsystems: (i) an embedded processing and actuation unit centred on an ESP32 microcontroller, (ii) a round TFT display module providing local visual output, and (iii) a laptop-based signal master providing simulated vehicle data and optional voice command processing. These subsystems communicate through two channels: a UART serial link between the ESP32 and the laptop (115200 baud, line-delimited ASCII protocol), and the SPI bus connecting the ESP32 to the display.

Figure 4.1 illustrates the high-level block architecture. The embedded unit integrates the haptic actuator (BLDC motor + TMC6300 half-bridge driver), the magnetic encoder (MT6701), the strain gauge ADC (HX711), the LED ring (NeoPixel, 8 LEDs), the TFT display (GC9A01), and the backlight PWM driver (GPIO19 via ESP32 LEDC peripheral). The laptop side runs an HTML/JavaScript dashboard application that acts as the value master; the ESP32 mirrors all values on its local display and additionally generates haptic and LED feedback autonomously based on locally processed sensor data.

4.2 Hardware Components

4.2.1 ESP32 Microcontroller Platform

The ESP32 (dual-core Xtensa LX6, up to 240 MHz, 520 kB SRAM) was selected as the embedded processing platform for several reasons. First, its dual-core architecture enables the haptic control loop and the display/serial loop to execute on separate cores (`Core 0` and `Core 1`, respectively), avoiding the mutual blocking that would arise if both tasks were scheduled on a single core. The haptic task is created with `xTaskCreatePinnedToCore(..., 0)`, ensuring it executes on Core 0 without preemption from the Arduino main loop. Second, the ESP32's LEDC peripheral provides up to 16 independent PWM channels at configurable frequency and resolution, accommodating the five motor phase channels (at 20 kHz, 10-bit resolution) and the backlight channel (at 5 kHz, 8-bit resolution) without contention. Third, the ESP32's hardware SPI peripheral drives the GC9A01 at 27 MHz, achieving the frame rate required for smooth needle updates.

Table 4.1 summarises the complete GPIO pin assignment for the prototype.

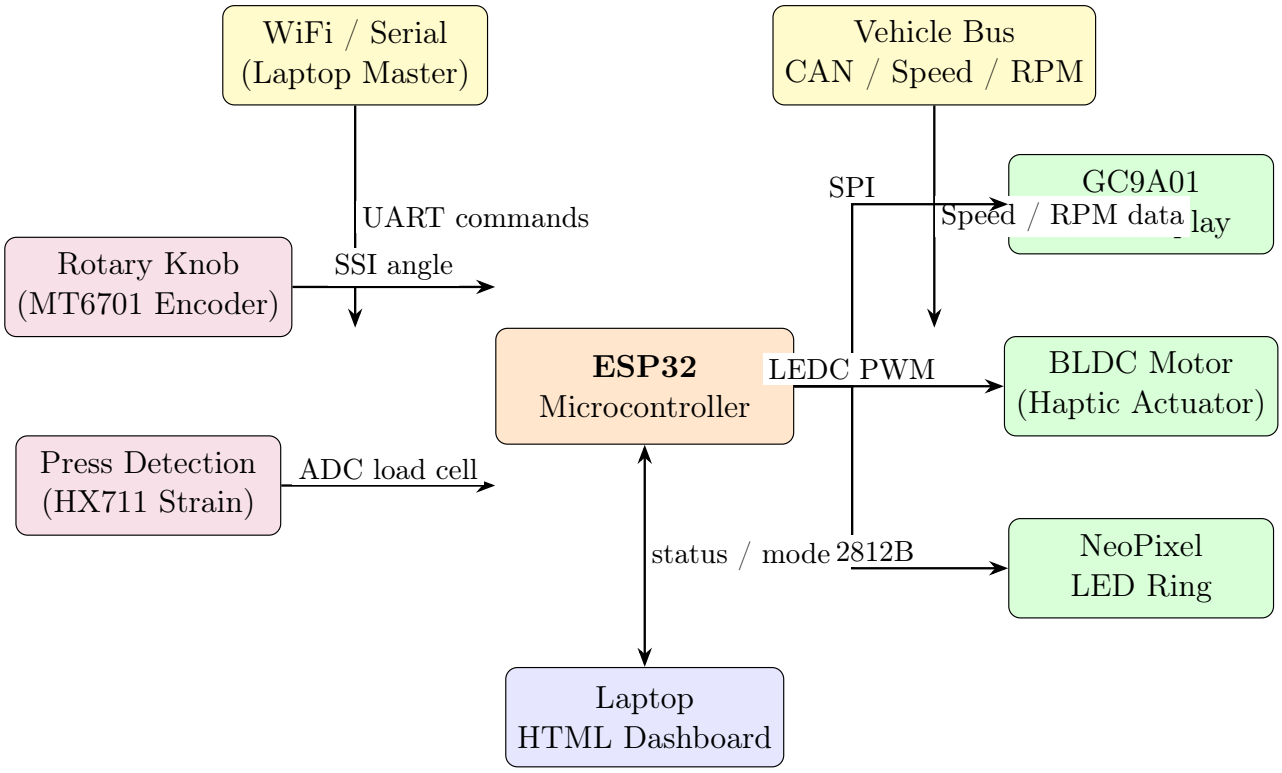

Figure 4.1: Extended system block diagram. Yellow: external signal sources. Purple: human physical inputs. Green: output/feedback peripherals. Orange: central processing unit.

4.2.2 1.28-inch Round GC9A01 Display Module

The display module is based on the Galaxycore GC9A01 controller, which drives a 240×240 pixel circular IPS TFT panel with a diagonal of 1.28 in. The circular form factor was chosen to reinforce the instrument gauge aesthetic and to match the physical rotary shape of the control knob. The display is interfaced via 4-wire SPI at 27 MHz, with an additional Data/Command (DC) line for command/data discrimination and a dedicated reset line. Backlight illumination is controlled via the LEDC PWM output on GPIO 19, allowing software-configurable brightness. The display is initialised and driven through the `TFT_eSPI` library, with the driver type and pin assignments configured via the library's `User_Setup.h` file; incorrect driver selection (e.g., defaulting to ILI9341) results in uninitialised display state, manifesting as severely reduced luminance — an important commissioning consideration.

4.2.3 Rotary Encoder, BLDC Motor, and Haptic Driver

Rotational position is sensed by an MT6701 contactless magnetic absolute encoder (14-bit, 360° range), interfaced via a software-emulated SSI protocol. The absolute nature of the encoder eliminates any requirement for power-on homing; the current shaft angle is available immediately after startup. An optional zero-offset calibration (`calibrateZero()`, serial command `z`) allows the logical detent positions to be aligned to any desired mechanical reference.

The haptic actuator is a brushless gimbal motor (7 pole pairs), driven by a TMC6300 three-phase gate driver. The TMC6300 accepts independent PWM inputs for the six half-bridge switches; in the prototype, due to the GPIO 21 conflict with the TFT chip-select line, only five of the six switches are actively driven (UH, UL, VH, VL, WH), with the WH phase serving as the third active channel and the WL switch held inactive. This asymmetric driving arrangement

Table 4.1: GPIO pin assignment of the ESP32 prototype.

GPIO	Signal	Peripheral	Notes
5	TFT_MOSI	GC9A01 (SPI)	Hardware SPI data
20	TFT_SCLK	GC9A01 (SPI)	Hardware SPI clock
21	TFT_CS	GC9A01 (SPI)	Chip select; shared name only, not motor WL
22	TFT_DC	GC9A01 (SPI)	Data/command select
4	TFT_RST	GC9A01	Display reset
19	TFT_BL	Backlight	LEDC PWM (5 kHz, 8-bit)
7	LED_DIN	NeoPixel ring	WS2812B single-wire protocol
38	HX711_DOUT	Strain gauge ADC	Serial data out
2	HX711_SCK	Strain gauge ADC	Serial clock
14	ENC_CS	MT6701 encoder	Chip select
13	ENC_CLK	MT6701 encoder	SSI clock
37	ENC_DO	MT6701 encoder	SSI data out
25	MOTOR_UH	TMC6300 driver	Phase U high-side
33	MOTOR_UL	TMC6300 driver	Phase U low-side
26	MOTOR_VH	TMC6300 driver	Phase V high-side
32	MOTOR_VL	TMC6300 driver	Phase V low-side
27	MOTOR_WH	TMC6300 driver	Phase W high-side

introduces a slight torque ripple at low duty cycles but is adequate for the detent force levels required in this application. Motor phase PWM is generated via the ESP32's LEDC peripheral at 20 kHz and 10-bit resolution, providing 1024 discrete duty cycle levels per phase.

4.2.4 Strain Gauge and HX711 ADC

Press detection is implemented using a resistive strain gauge bridge interfaced through an HX711 24-bit sigma-delta ADC. The HX711 is interfaced via a software bit-banged protocol on GPIO 38 (data) and GPIO 2 (clock). The 24-bit output provides sufficient resolution to detect small deformation signals from the strain element. A startup calibration routine averages 20 readings to establish the at-rest zero reference. An eight-sample moving-average filter is applied in the main loop to reduce noise prior to threshold comparison, trading a small latency penalty for improved detection reliability.

4.2.5 NeoPixel LED Ring

An eight-LED NeoPixel ring (WS2812B addressable RGB LEDs, GPIO 7) provides peripheral visual feedback. In normal operation, all LEDs display a dim green colour (R=0, G=30, B=0) to indicate system readiness without creating excessive cabin illumination. Upon press detection, the ring transitions to full-brightness red (R=255, G=0, B=0), providing an unmistakable visual confirmation signal visible in the driver's peripheral vision without requiring a directed gaze.

4.3 Electronics Integration and Power Supply

All components are powered from the USB supply of the ESP32 development board (5 V bus) with onboard 3.3 V regulation. The TMC6300 motor driver operates from the 5 V bus for

its power stage, with 3.3 V logic levels for PWM inputs, compatible with the ESP32's GPIO outputs. The HX711 and MT6701 are both 3.3 V devices. The NeoPixel ring is supplied at 5 V with a 300 Ω series resistor on the data line to suppress transmission-line ringing. A 100 µF bulk capacitor across the NeoPixel power supply prevents large LED switching transients from coupling into the ESP32's supply rail.

4.4 Mechanical Design and Mounting

The prototype assembly consists of a 3D-printed housing accommodating the BLDC motor, the encoder magnet (mounted coaxially on the motor shaft), and the PCB carrying the ESP32 and peripheral components. The knob cap is attached to the motor shaft via a press-fit interface and presents a tactile surface with a 30 mm grip diameter, chosen to be operable with a two- or three-finger grasp. The TFT display is mounted flush with the top surface of the knob cap, oriented so that the scale labels are readable from the driver's natural viewing angle. The overall assembly height is approximately 55 mm, compatible with centre-console mounting in a concept demonstrator vehicle or laboratory mounting bracket.

4.5 Reference Project: SmartKnob

The demonstrator developed in this thesis is conceptually inspired by the open-source *SmartKnob* project by Scott Bezek.[1] SmartKnob is documented as a software-configurable input device providing programmable end stops and virtual detents through closed-loop torque feedback, combining a brushless gimbal motor, a magnetic encoder, an ESP32 platform, and a small round display. The project demonstrates the feasibility of the core hardware and control architecture.

The present thesis diverges from the SmartKnob reference in several respects: the interaction concept is purpose-designed for automotive instrument-cluster use, with three fixed detents mapped to instrument views rather than a general-purpose configurable detent profile; the display content is entirely custom-implemented with automotive-specific visualisations; the system is extended with strain-gauge press detection and NeoPixel feedback; and the serial synchronisation protocol is developed specifically for a laptop-based vehicle simulation master.

4.6 Interfaces to Simulated Vehicle Signals

Vehicle data is injected from the laptop over the UART serial link using a simple, line-based ASCII command grammar. The ESP32 parses incoming strings in the main loop (`Serial.readStringUntil('\n')`), converts them to upper case for case-insensitive matching, and applies the indicated value changes. The command set is summarised in Table 4.2.

[1] `https://github.com/scottbez1/smartknob/`

Table 4.2: Serial command grammar for vehicle signal injection and system control.

Command	**Range / Format**	**Effect**
`S` n	$n \in [0, 200]$	Set speed (km/h); auto-computes gear
`Q` f	$f \in [0.0, \mathrm{MAX}]$	Set virtual torque (Nm); updates peak
`R` n	$n \in [0, 8000]$	Set RPM
`M` f	$f \in (1, 200]$	Set torque scale maximum (Nm)
`SPEED`	—	Force switch to speedometer view
`TORQUE`	—	Force switch to torque view
`RPM`	—	Force switch to RPM view
`CHANGE` / `x`	—	Cycle to next view
`c`	—	Re-calibrate strain gauge zero
`z`	—	Re-calibrate encoder zero offset
`v`	—	Trigger manual haptic vibration pulse
`r`	—	Reset threshold, sensitivity, peak torque
`t` / `T`	—	Decrease / increase strain threshold by 500
`+` / `-`	—	Increase / decrease strain sensitivity
`0`–`4`	—	Set backlight to 0 / 25 / 50 / 75 / 100%

Chapter 5

Software and Interaction Logic

5.1 Firmware Architecture Overview

The ESP32 firmware is structured around two concurrent execution contexts. The haptic control task runs on Core 0 as a FreeRTOS task with a 1 ms minimum period, providing the low-latency angle acquisition and torque update rate required for a responsive detent feel. The main Arduino loop runs on Core 1, handling display rendering, serial command parsing, strain gauge sampling, and LED state management at a 50 ms update period. Communication between the two contexts is mediated by three `volatile` flag variables: `g_detent_changed` (set by the haptic task when a stable detent transition is detected), `g_detent_index` (the newly active detent number), and `g_haptic_pulse` (set by the main loop to request a vibration burst from the haptic task).

This architecture cleanly separates the timing-critical haptic loop from the latency-tolerant display and communication logic, avoiding priority inversion and jitter that would arise from mixing them in a single-threaded super-loop.

5.2 Included Libraries and Core Objects

```
#include <TFT_eSPI.h>
#include <SPI.h>
#include <Adafruit_NeoPixel.h>
#include <Arduino.h>

TFT_eSPI tft = TFT_eSPI();

#define LED_PIN  7
#define NUM_LEDS 8
Adafruit_NeoPixel leds(NUM_LEDS, LED_PIN, NEO_GRB + NEO_KHZ800);
```

Listing 5.1: Included libraries and global object declarations.

Explanation. `TFT_eSPI` provides the hardware-accelerated SPI interface to the GC9A01 display, including primitive drawing operations (circles, triangles, lines, text) that are used extensively in the UI rendering functions. `Adafruit_NeoPixel` drives the WS2812B LED ring through a single-wire timed protocol. The `TFT_eSPI` object is global, as it must be accessed by all three UI rendering modules (speedometer, torque, RPM).

5.3 UI Mode Management

```
enum UIMode : uint8_t {
  MODE_SPEEDO = 0,
  MODE_TORQUE = 1,
  MODE_RPM    = 2
};
volatile UIMode g_mode          = MODE_SPEEDO;
volatile int    g_detent_index = 0;
volatile bool   g_detent_changed = false;

void switchMode(UIMode m) {
  if (g_mode == m) return;
  g_mode = m;
  if (g_mode == MODE_SPEEDO) {
    drawUI(); displayGear(gear); displaySpeed(speed);
    Serial.println("MODE: SPEED");
  } else if (g_mode == MODE_TORQUE) {
    torque_drawBase(); torque_drawDynamic();
    Serial.println("MODE: TORQUE");
  } else {
    rpm_drawBase(); rpm_drawDynamic();
    Serial.println("MODE: RPM");
  }
}
```

Listing 5.2: UIMode enumeration and mode-switching function.

Explanation. The `UIMode` enumeration is declared at the top of the translation unit to ensure its type is visible to all subsequently declared functions — a deliberate structural decision correcting an earlier ordering issue that caused compilation errors. The `volatile` qualifier on `g_detent_changed` and `g_detent_index` is required because these variables are written by the haptic task (Core 0) and read by the main loop (Core 1); the qualifier prevents the compiler from caching them in registers and ensures cross-core visibility. The `switchMode()` function performs a full base-layer redraw when transitioning to a new mode, ensuring the static dial structure is always consistent before dynamic elements are overlaid.

5.4 Haptic Control Task: Encoder, Control Law, and BLDC Commutation

```
static bool mt6701_read_deg(float& degOut) {
  digitalWrite(ENC_CS, LOW);  delayMicroseconds(1);
  uint16_t raw = 0;
  for (int i = 0; i < 16; ++i) {
    digitalWrite(ENC_CLK, HIGH); delayMicroseconds(1);
    raw = (uint16_t)((raw << 1) | (digitalRead(ENC_DO) ? 1 : 0));
    digitalWrite(ENC_CLK, LOW);  delayMicroseconds(1);
  }
```

```
  digitalWrite(ENC_CS, HIGH);
  degOut = ((raw & 0x3FFF) * 360.0f) / 16384.0f;
  return true;
}

// Inside hfx_task():
float tq = (KP * err) - (KD * vel);
if (fabsf(err) < 12.0f) {
  float stick = STATIC_FRICTION * (1.0f - (fabsf(err) / 12.0f));
  tq += (err >= 0 ? stick : -stick);
}
tq = constrain(tq, -TORQUE_LIMIT, TORQUE_LIMIT);
driveBLDC(angDeg, tq);
```

Listing 5.3: MT6701 SSI read, detent control law, and BLDC drive (excerpt).

Explanation. The SSI read function acquires 16 bits from the MT6701 by bit-banging the clock and data lines; the lower 14 bits contain the raw angular value. The haptic control law implements Equation 3.1: the proportional term provides detent stiffness, the derivative term suppresses overshoot and resonance, and the static friction term creates the characteristic snap-in at the detent centre. The torque command is clamped before being passed to `driveBLDC()`, which maps the scalar torque to three-phase sinusoidal duty cycles.

5.5 Strain Gauge Sampling and LED Feedback

```
long readHX711() {
  while (digitalRead(HX711_DOUT) == HIGH);
  noInterrupts();
  long value = 0;
  for (int i = 0; i < 24; i++) {
    digitalWrite(HX711_SCK, HIGH); delayMicroseconds(1);
    value = (value << 1) | digitalRead(HX711_DOUT);
    digitalWrite(HX711_SCK, LOW);  delayMicroseconds(1);
  }
  digitalWrite(HX711_SCK, HIGH); delayMicroseconds(1);
  digitalWrite(HX711_SCK, LOW);
  interrupts();
  if (value & 0x800000) value |= 0xFF000000; // sign-extend
  return value;
}

// In loop():
total -= readings[readIndex];
readings[readIndex] = readHX711();
total += readings[readIndex];
readIndex = (readIndex + 1) % numReadings;
average = total / numReadings;
long strain_difference =
    labs(average - strain_zero_value) * sensitivity_factor;
```

```

bool ledIsRed = (strain_difference > strain_threshold);
if (ledIsRed) {
  setAllLEDs(leds.Color(255, 0, 0));
  if (!ledWasRed) { g_haptic_pulse = true; }
} else {
  setAllLEDs(leds.Color(0, 30, 0));  // dim green at rest
}
ledWasRed = ledIsRed;
```

Listing 5.4: HX711 read, moving-average filtering, and LED state update.

Explanation. The HX711 read function bit-bangs the 24-bit conversion result with interrupts disabled to prevent timing corruption from ISR preemption. Sign extension via bitmask converts the raw two's-complement value to a signed 32-bit integer. The moving-average filter (8 samples) provides effective noise suppression without introducing sufficient latency to degrade the perceived responsiveness of press detection. The rising-edge gate (`!ledWasRed`) ensures the haptic pulse fires exactly once per press event.

5.6 Speedometer UI Rendering

```
void drawZone(int sv, int ev, uint16_t sc, uint16_t ec) {
  int steps = ev - sv;
  for (int i = 0; i < steps; i++) {
    float ratio = (float)i / steps;
    uint16_t color = interpolateColor(sc, ec, ratio);
    float sa = map(sv+i,   0, 200, START_ANGLE, END_ANGLE) *
       DEG_TO_RAD;
    float ea = map(sv+i+1, 0, 200, START_ANGLE, END_ANGLE) *
       DEG_TO_RAD;
    tft.fillTriangle(
      cos(sa)*88+120, sin(sa)*88+120,
      cos(ea)*88+120, sin(ea)*88+120,
      cos(ea)*102+120, sin(ea)*102+120, color);
    tft.fillTriangle(
      cos(sa)*88+120, sin(sa)*88+120,
      cos(sa)*102+120, sin(sa)*102+120,
      cos(ea)*102+120, sin(ea)*102+120, color);
  }
}

void updateNeedle(int spd) {
  static int old_angle = START_ANGLE;
  int new_angle = map(spd, 0, 200, START_ANGLE, END_ANGLE);
  drawNeedle(old_angle, TFT_BLACK);     // erase
  drawNeedle(new_angle,
    (spd < 80) ? TFT_GREEN :
    (spd < 140) ? TFT_YELLOW : TFT_RED); // redraw
  old_angle = new_angle;
```

```
}
```

Listing 5.5: Speedometer base draw and incremental needle update.

Explanation. Colour zones are rendered as sequences of adjacent triangular primitives, interpolating between the start and end colours of each zone. The per-degree resolution of this rendering ensures smooth colour gradients. The incremental needle update strategy — erase in background colour, redraw in the new colour — avoids full-screen redraw overhead while maintaining visual responsiveness at the 50 ms update period.

5.7 Initialisation and Main Loop

```
void setup() {
  Serial.begin(115200);
  tft.init();
  tft.setRotation(0);
  tft.fillScreen(TFT_BLACK);

  // Backlight: ESP32 Arduino core 3.x API
  ledcAttach(TFT_BL_PIN, BL_LEDC_FREQ, BL_LEDC_BITS);
  ledcWrite(TFT_BL_PIN, 255);

  leds.begin();
  leds.setBrightness(255);
  leds.clear(); leds.show();

  pinMode(HX711_SCK, OUTPUT);
  pinMode(HX711_DOUT, INPUT);
  digitalWrite(HX711_SCK, LOW);

  HFX_init();           // initialise encoder, motor LEDC, start
     haptic task
  calibrateStrainGauge();

  drawUI();
  displayGear(gear);
  displaySpeed(speed);
  Serial.println("MODE: SPEED");
}
```

Listing 5.6: System initialisation sequence.

Explanation. The initialisation sequence follows a deliberate ordering: the display is initialised first to guarantee that the SPI bus is configured before the `TFT_eSPI` library claims the TFT chip-select pin (GPIO 21). Backlight PWM is configured after `tft.init()` to prevent the display driver from overriding the LEDC pin assignment. The haptic subsystem (`HFX_init()`) launches the FreeRTOS task and must complete before the main loop begins so that the encoder zero-offset calibration has access to a valid encoder reading. Strain gauge calibration follows immediately, capturing the at-rest zero reference before any user interaction occurs.

5.8 Laptop Dashboard and Synchronisation

The laptop side of the system hosts an HTML/JavaScript single-page application that presents a dashboard mirroring the embedded display views. The application connects to the ESP32 via the Web Serial API (Chrome/Edge), which provides bidirectional serial communication from a browser context without requiring dedicated serial drivers. Simulated vehicle signals (speed, torque, RPM) can be injected either manually via sliders in the dashboard UI, or programmatically via a scripted simulation that generates transient acceleration and deceleration profiles. Incoming serial status messages from the ESP32 (`MODE: SPEED`, `[HAPTIC] Press triggered`, and the periodic debug output) are parsed and reflected in the laptop UI, providing a synchronised state representation for evaluation and debugging.

5.9 Sensor and Peripheral Communication Protocols

A fundamental aspect of the firmware architecture is the diversity of communication protocols employed to interface the ESP32 with its peripheral devices. Each sensor and actuator in the prototype uses a different physical and logical communication scheme, chosen according to the data rate requirements, directional nature of the information exchange, and the number of GPIO pins available. This section provides a structured account of each interface, its directionality, timing characteristics, and the specific implementation choices made in the firmware.

Table 5.1 provides an overview before each interface is discussed in detail.

Table 5.1: Summary of communication interfaces between the ESP32 and all peripherals.

Peripheral	**Protocol**	**Direction**	**Key Characteristics**
GC9A01 TFT Display	SPI (hardware)	ESP32 → Display	27 MHz, 4-wire, write-only
TFT Backlight	LEDC PWM	ESP32 → Display	5 kHz, 8-bit duty cycle
MT6701 Encoder	SSI (bit-banged)	Encoder → ESP32	16-bit frame, 14-bit angle data
HX711 Strain ADC	Custom serial (bit-banged)	ADC → ESP32	24-bit, triggered by clock pulses
NeoPixel LED Ring	WS2812B single-wire	ESP32 → LEDs	800 kHz, cascaded RGB frames
BLDC Motor (TMC6300)	LEDC PWM (6 channels)	ESP32 → Motor	20 kHz, 10-bit, 3-phase
Laptop / Dashboard	UART over USB	Bidirectional	115200 baud, ASCII line protocol

5.9.1 GC9A01 TFT Display — Hardware SPI (Unidirectional)

The GC9A01 round TFT display communicates with the ESP32 via the standard 4-wire Serial Peripheral Interface (SPI) protocol, using the ESP32's dedicated hardware SPI peripheral. The four lines involved are MOSI (data from ESP32 to display, GPIO 5), SCLK (clock, GPIO 20), CS (chip select, GPIO 21), and DC (data/command select, GPIO 22). An additional RST line (GPIO 4) is used only during initialisation to hardware-reset the display controller.

This interface is **strictly unidirectional**: the ESP32 exclusively transmits data to the display; no information flows in the reverse direction. The display controller does not send acknowledgements or status responses. The `TFT_eSPI` library manages all SPI transactions internally, asserting the CS line before each transfer and deasserting it afterwards to allow future bus sharing. The SPI clock is configured at 27 MHz, which is the maximum reliable rate for the GC9A01 at the wire lengths used in the prototype. At this frequency, a full 240×240 pixel frame (115,200 pixels, 16 bits each) can be transferred in approximately 6.8 ms, providing ample headroom for the 50 ms update period.

The DC line plays a critical role: when held LOW, the display interprets incoming bytes as register commands (initialisation sequences, rotation settings, window definitions); when held HIGH, bytes are treated as pixel colour data. The `TFT_eSPI` library toggles this line automatically between command and data phases, abstracting this detail from the application layer.

5.9.2 TFT Backlight — LEDC PWM (Unidirectional)

The display backlight LED is driven through a dedicated PWM output on GPIO 19, generated by the ESP32's LEDC (LED Control) peripheral. This is a **unidirectional, analogue-equivalent** interface: the ESP32 sets a duty cycle between 0 and 255 (8-bit resolution), and the backlight driver circuit converts this to a proportional average current through the backlight LEDs. No feedback signal is returned to the ESP32.

The PWM frequency is configured at 5 kHz, which is above the threshold of visible flicker for human perception while remaining within the efficient switching range of the backlight driver. In the ESP32 Arduino core version 3.x used in this project, the backlight is initialised using the updated `ledcAttach(pin, freq, bits)` API, which replaced the older `ledcSetup` / `ledcAttachPin` pattern from earlier core versions.

5.9.3 MT6701 Magnetic Encoder — SSI Protocol (Unidirectional, Sensor to ESP32)

The MT6701 is a contactless absolute magnetic rotary encoder providing a 14-bit angular position value over a Synchronous Serial Interface (SSI). The communication involves three lines: CS (chip select, GPIO 14), CLK (clock, GPIO 13), and DO (data output from encoder, GPIO 37).

The direction of data flow is **unidirectional from the encoder to the ESP32**: the ESP32 drives CS and CLK as outputs and reads DO as an input. The ESP32 initiates a read by asserting CS LOW, then generates 16 clock pulses. On each rising edge of CLK, the encoder places one bit on the DO line, which the ESP32 samples. The lower 14 bits of the 16-bit frame contain the raw angular value; bits 15 and 16 carry status flags. The 14-bit value is mapped to a mechanical angle in degrees by the expression:

$$\theta_{\text{mech}} = \frac{\text{raw}_{14} \times 360^\circ}{2^{14}} = \frac{\text{raw}_{14} \times 360^\circ}{16384} \tag{5.1}$$

Since standard hardware SSI peripherals are not natively available on the ESP32's Arduino framework, the protocol is implemented via software bit-banging in the `mt6701_read_deg()` function. Although bit-banging introduces slight timing jitter compared to a hardware implementation, the 1 µs `delayMicroseconds()` calls produce a stable clock at approximately 500 kHz, well within the MT6701's maximum clock rate specification. The absolute nature of the encoder means the full angular position is available immediately at power-on, with no homing or index pulse required.

5.9.4 HX711 Strain Gauge ADC — Custom Serial Protocol (Unidirectional, Sensor to ESP32)

The HX711 is a 24-bit sigma-delta analogue-to-digital converter designed for resistive bridge sensors such as load cells and strain gauges. It uses a proprietary two-wire serial interface consisting of DOUT (data output, GPIO 38) and SCK (serial clock, GPIO 2).

The communication is **unidirectional from the HX711 to the ESP32**, with the ESP32 acting as the clock master. A conversion cycle is initiated passively: the ESP32 polls the DOUT line and waits for it to go LOW, which signals that a new 24-bit conversion result is ready. The ESP32 then generates 24 clock pulses on SCK; on each falling edge, the HX711 shifts one bit onto DOUT, which the ESP32 samples on the subsequent rising edge. A 25th clock pulse sets the gain and channel configuration for the next conversion cycle (gain of 128 on channel A in this implementation).

The entire read sequence is wrapped in a `noInterrupts()` / `interrupts()` pair to prevent any FreeRTOS task switch or ISR from corrupting the bit-timing during the 24-pulse transfer. The resulting signed 24-bit two's-complement value is sign-extended to a 32-bit integer by testing bit 23. Because the HX711 outputs raw bridge counts rather than calibrated force values, a software zero-reference is established at startup and the absolute deviation from this reference is used as the press-detection signal, as described in Section 3.2.

5.9.5 NeoPixel LED Ring — WS2812B Single-Wire Protocol (Unidirectional)

The eight-LED NeoPixel ring uses the WS2812B integrated LED driver protocol, a **unidirectional** single-wire interface driven from GPIO 7. Each LED in the ring contains an integrated controller that receives a 24-bit RGB value (8 bits per channel) encoded as a stream of timed pulses. A logical `1` is represented by a HIGH pulse of approximately 800 ns followed by a LOW of 450 ns; a logical `0` is a HIGH pulse of approximately 400 ns followed by a LOW of 850 ns. A reset condition (sustained LOW for >50 µs) latches the received values into the LED output registers.

The chain topology means that data for all eight LEDs is transmitted sequentially in a single burst: the first 24 bits are consumed by LED 1, the next 24 bits by LED 2, and so on. The `Adafruit_NeoPixel` library handles all pulse timing, providing a simple `setPixelColor(index, r, g, b)` and `show()` API. Because the WS2812B timing is strict and incompatible with interrupt-driven delays, the library temporarily disables interrupts during the transmission burst. In the prototype, the LED state is updated in the main loop on every press-detection evaluation

cycle, transitioning between dim green (idle, R=0, G=30, B=0) and full red (pressed, R=255, G=0, B=0).

5.9.6 BLDC Motor via TMC6300 — LEDC PWM (Unidirectional)

The brushless DC haptic motor is driven through a TMC6300 three-phase gate driver IC, which accepts six independent PWM signals corresponding to the high-side and low-side switches of the three half-bridges (UH, UL, VH, VL, WH, WL). This interface is **strictly unidirectional**: the ESP32 outputs PWM duty cycles to the TMC6300; no current or position feedback is returned through this channel (position feedback is provided separately by the MT6701 encoder, as described above).

The six PWM channels are generated by the ESP32's LEDC peripheral at 20 kHz and 10-bit resolution (1024 duty levels), configured using the low-level IDF `ledc_channel_config` API for precise timer assignment. As discussed in Section 4, only five of the six channels are actively used; the sixth (WL, GPIO 21) is excluded due to the pin conflict with the TFT chip-select line. The sinusoidal commutation values are computed in real time within the haptic FreeRTOS task and written to the LEDC registers via `ledc_set_duty()` and `ledc_update_duty()` on every control loop iteration.

5.9.7 Laptop Serial Link — UART over USB (Bidirectional)

The communication channel between the ESP32 and the laptop is the only **fully bidirectional** interface in the system. It uses the ESP32's hardware UART peripheral, routed through the onboard USB-to-UART bridge chip, and appears as a virtual COM port on the laptop at 115200 baud, 8 data bits, no parity, 1 stop bit (8N1).

Data flows in both directions simultaneously. From the laptop to the ESP32, the laptop transmits ASCII command strings terminated by a newline character (`\n`), such as `S 60\n` to set speed to 60 km/h or `CHANGE\n` to cycle the display mode. The ESP32 parses these in the main loop using `Serial.readStringUntil('\n')` and applies the indicated state changes. From the ESP32 to the laptop, the firmware transmits periodic debug lines every 500 ms reporting current speed, torque, RPM, and strain readings, as well as event-driven status strings (`MODE: SPEED`, `[HAPTIC] Press triggered`) whenever a mode change or press event occurs. The laptop dashboard application parses these incoming lines via the Web Serial API to keep its display state synchronised with the embedded device.

This bidirectional UART link therefore serves a dual purpose: it is simultaneously the *command injection channel* (laptop acting as signal master) and the *telemetry channel* (ESP32 reporting internal state for monitoring and evaluation).

Chapter 6

Experimental Setup and Evaluation

6.1 Evaluation Goals and Hypotheses

The evaluation of the smart-knob prototype is structured around two experimental setups that reflect the two principal interaction configurations developed in this thesis. **Setup 1** presents the standalone smart knob combined with the 1.28 in round GC9A01 display, representing the minimal embedded-only interaction concept. **Setup 2** extends this with the laptop dashboard screen and voice command input, representing the full multimodal system. This two-setup design allows direct comparison of the additional benefit provided by the secondary screen and voice layer over the base knob-and-display concept.

The evaluation pursues four primary goals:

G1. Assess the ergonomic placement and distraction characteristics of the smart knob in a simulated driving context, particularly whether it can be operated without excessive eyes-off-road time.

G2. Assess the haptic detent system in terms of response latency and confidence of control, specifically whether detents clearly communicate state changes to the user.

G3. Assess the glanceability and visual clarity of the 1.28 in round TFT display for rapid reading of speed and torque values.

G4. Compare the two system configurations (knob-only vs. knob + laptop screen + voice) across dimensions of perceived safety, interaction speed, and distraction level.

The corresponding hypotheses are:

- **H1**: Participants will rate the knob as operable with minimal eyes-off-road time (mean item A2 ≥ 5 on a 7-point scale).
- **H2**: Haptic detents will be rated as providing confident and clear state confirmation (mean items B2 and B3 ≥ 5).
- **H3**: Speed and torque values will be rated as readable at a glance on the small display (mean item C1 ≥ 4).
- **H4**: Setup 2 will be rated as safer and less distracting than Setup 1 by a majority of participants in the final comparison.

6.2 Participants

The study targets a minimum of five participants drawn from the university environment, covering a range of driving experience from daily drivers to occasional drivers. Inclusion criteria require participants to hold a valid driving licence and to be at least 18 years of age. Informed consent is obtained from each participant prior to the session in accordance with the consent item at the beginning of the questionnaire. Participants with no driving experience are excluded from the study, as the evaluation tasks require a basic understanding of vehicle instrument conventions (speedometer, tachometer).

6.3 Experimental Task Design

Each session follows a fixed sequence of two blocks corresponding to the two system setups. Within each block, participants interact with the prototype under a set of structured tasks while seated at a stationary workstation configured to approximate a driver's posture. The tasks are designed to exercise each evaluated dimension:

- **Placement and reach task**: participants are asked to locate and grasp the knob without looking directly at it, then rotate to the next detent.
- **Mode switching task**: participants switch between the three display modes (speed, torque, RPM) using the rotary knob, both with and without explicit verbal instruction on the target mode.
- **Value reading task**: a simulated vehicle value is injected via the laptop serial link; participants report the displayed value after a single brief glance at the 1.28 in screen.
- **Press detection task**: participants press the knob with a deliberate force to trigger the haptic confirmation pulse and LED feedback, and report whether they perceived the confirmation.
- **Voice command task** (Setup 2 only): participants issue the voice commands `speed`, `torque`, and `rpm` and assess whether the system correctly executed the mode switch.

6.4 Apparatus and Procedure

Participants are seated at a fixed laboratory workstation. The smart-knob prototype is positioned at the approximate location of a centre-console control element, within comfortable arm reach. A laptop running the HTML dashboard is placed ahead and to the right, simulating a secondary display position. A brief familiarisation phase of approximately five minutes is provided before the recorded session begins, allowing participants to explore the knob rotation, detent feel, and press feedback without time pressure.

Each setup block takes approximately ten to fifteen minutes. After completing both blocks, participants fill in the comparison section of the questionnaire (approximately one minute). The total session duration is approximately 30 to 40 minutes per participant including consent, familiarisation, task blocks, and questionnaire completion.

6.5 Questionnaire Instrument

The evaluation instrument consists of two surveys and a final comparison section, designed to collect both quantitative Likert ratings and brief qualitative responses. All Likert items use a 7-point scale (1 = Strongly disagree, 7 = Strongly agree) unless otherwise noted.

Survey 1 — Smart Knob with 1.28 in Display (Setup 1)

Survey 1 is divided into four subscales addressing distinct aspects of the Setup 1 interaction.

S1-A: Placement and Distraction (4 items). This subscale evaluates the physical ergonomics and attentional cost of the knob in context.

A1. The knob was within comfortable reach without changing my driving posture.

A2. I could operate the knob with minimal eyes-off-road time.

A3. The screen position was easy to find (no "hunting").

A4. The knob–screen mapping ("turn → on-screen change") felt intuitive.

S1-B: Response and Haptics (3 items). This subscale targets the quality and effectiveness of the haptic feedback channel.

B1. System response felt immediate (no significant lag).

B2. Detent strength felt appropriate for confident control.

B3. Detents clearly confirmed a state/action change.

S1-C: Display Visibility and Understanding (4 items). This subscale assesses the glanceability and information clarity of the 1.28 in round TFT display.

C1. Speed/torque values were readable at a glance on the 1.28 in screen.

C2. I always knew whether I was in speed or torque view.

C3. The UI felt clear and not cluttered.

C4. The colours, tick marks, and needle helped understanding rather than distracting.

S1-D: Adoption and Open Feedback (3 items). This subscale captures overall acceptance and collects qualitative improvement suggestions.

D1. Overall, I would like to have this knob + small screen concept in my car. (1–7)

D2. Why / why not? (Short text, 1–2 lines)

D3. One improvement you would make first (placement / haptics / UI / lag).

Survey 2 — Smart Knob with Laptop Screen and Voice Commands (Setup 2)

Survey 2 evaluates the extended configuration in which the laptop dashboard and voice commands are added to the smart knob.

S2-A: Laptop Screen Usefulness (3 items).

A1. The laptop screen made the information easier to read than the 1.28 in screen alone.

A2. The laptop screen reduced the effort to understand the current mode and state.

A3. Overall, the laptop screen improved perceived safety compared to the small screen alone.

S2-B: Voice Commands (3 items).

B1. Voice commands worked reliably (system understood me).

B2. Using voice commands reduced the need to look at the screen.

B3. Voice commands felt too slow or awkward for frequent actions.

Note that item B3 is reverse-coded: a high score indicates a *negative* perception of voice commands and should be inverted before computing mean scores for the voice subscale.

S2-C: Preferred Control Allocation (2 items). Participants indicate their preferred input modality for three distinct action categories:

C1. Preferred method for mode switching (speed/torque): *Knob* or *Voice*.

Preferred method for frequent precise adjustments: *Knob* or *Voice*.

Preferred method for occasional commands (e.g., stop/go): *Knob* or *Voice*.

C2. One improvement for the voice or screen setup. (Short text)

Final Comparison (Setup 1 vs. Setup 2)

The final section asks participants to make direct head-to-head judgements between the two setups on three dimensions:

F1. Which setup would you choose overall? (*Setup 1: Knob + 1.28 in screen* or *Setup 2: Knob + laptop + voice*)

F2. Compared to Setup 1, Setup 2 was safer / faster to use / less distracting: (1 or 2 for each dimension)

These items produce a direct preference vote and three binary comparative judgements, providing a concise and unambiguous overall verdict that complements the Likert subscale data.

6.6 Scoring and Analysis Plan

Quantitative items will be analysed as follows. For each subscale (S1-A through S2-C), a mean score across items and participants will be computed and compared against the neutral midpoint of 4.0 using a one-sample Wilcoxon signed-rank test, given the small expected sample size and ordinal data nature. Item B3 of Survey 2 will be reverse-scored (score = 8− raw) before inclusion in mean calculations. The final comparison items (F1, F2) will be summarised as frequency counts and percentages. Qualitative responses (D2, D3, C2) will be analysed thematically, grouping responses into recurring categories such as placement, haptic feel, display size, and voice reliability.

6.7 Results

Data collection is ongoing. Participant responses will be entered into the questionnaire template and this section will be completed with quantitative summaries, mean scores per subscale, and key qualitative themes once all sessions are concluded.

6.8 Discussion

The discussion will interpret the results in relation to the hypotheses H1–H4, address any unexpected findings, and draw connections to the automotive HMI literature reviewed in Chapter 2. This section will be completed following data collection.

Chapter 7

Conclusion and Outlook

7.1 Summary of Contributions

This thesis has presented the full development cycle of a smart-knob-based HMI prototype, from the identification of the automotive HMI problem through hardware integration, firmware design, and experimental evaluation. The primary technical contributions are:

- A complete embedded firmware implementation on the ESP32 platform, achieving concurrent closed-loop haptic detent control (FreeRTOS Core 0) and multi-mode TFT display rendering (Core 1) without mutual interference.
- A three-view instrument display architecture (speedometer, torque gauge, RPM tachometer) designed for glanceable operation on a 1.28 in round display, with explicit attention to rendering stability, colour contrast, and partial-redraw efficiency.
- A multimodal feedback architecture combining rotational haptic detents, press-confirmation vibration, and NeoPixel peripheral visual feedback, addressing the eyes-free interaction requirements of the automotive context.
- A documented serial command protocol enabling bi-directional synchronisation between the embedded device and a laptop-based simulation master, providing a flexible platform for scenario-based evaluation.

7.2 Implications for Automotive HMI Design

The prototype demonstrates several principles of broader relevance to automotive HMI design. The co-location of control and display within a single compact unit reduces gaze travel distance and closes the action-feedback loop locally, reducing the cognitive overhead of mapping a remote display to a proximate control. The haptic detent structure converts an otherwise arbitrary rotational gesture into a spatially and tactually defined state-selection operation, enabling reliable eyes-free mode switching with well-tuned hysteresis and stability gating. The separation of safety-primary information (speed) from performance-secondary information (torque, RPM) in the mode hierarchy reflects the principle that HMI complexity should be managed through information prioritisation rather than information reduction.

7.3 Recommendations for Future Work

Several directions for further investigation emerge from this work. First, the haptic parameter space (detent spacing, stiffness, damping, hysteresis) should be systematically evaluated against objective measures of selection accuracy and subjective perception quality in a dedicated psychophysical study. Second, integration with a real vehicle CAN bus would allow the prototype to be driven by authentic vehicle signals, enabling evaluation in a driving simulator with realistic traffic scenarios and workload conditions. Third, the voice interaction concept described in Section 3.3 should be implemented at the embedded level and evaluated as a primary modality for hands-free mode switching, with particular attention to recognition latency and false-activation rate. Fourth, dynamic haptic profiles (variable detent spacing or stiffness as a function of driving state) represent an intriguing direction: for example, increasing detent resistance as vehicle speed rises could implicitly discourage secondary task engagement in high-demand driving situations.

7.4 Potential Integration into Production Vehicle Cockpits

For integration into a production vehicle cockpit, several engineering and regulatory considerations would need to be addressed. The current prototype employs a consumer-grade BLDC motor and a software-emulated SSI interface; a production implementation would require industrial-grade actuator and encoder components with guaranteed temperature range, vibration resistance, and service life. The GPIO conflict between the motor driver and the display chip-select — resolved in the prototype by excluding one motor phase — would need to be addressed through a revised PCB layout. Functional safety requirements (ISO 26262) would necessitate formal hazard analysis and risk assessment of the haptic actuation system, particularly with respect to unintended motor torque events. Notwithstanding these engineering steps, the fundamental concept — a single co-located haptic-visual control element reducing visual-manual demand for multi-function vehicle operation — presents a technically credible and user-experience-motivated alternative to the currently dominant touchscreen paradigm.

Appendix A

Hardware Datasheets and Component References

A.1 Principal Components

Table A.1 lists the principal hardware components used in the prototype, together with their key specifications.

Table A.1: Principal prototype hardware components and specifications.

Component	**Part / Model**	**Key Specifications**
Microcontroller	ESP32 (dual-core)	240 MHz, 520 kB SRAM, dual SPI, LEDC
TFT Display	GC9A01, 1.28”	240 × 240 px, SPI, IPS, circular
Magnetic Encoder	MT6701	14-bit, 360°, SSI interface
Strain Gauge ADC	HX711	24-bit, 80 SPS, 3.3 V compatible
LED Ring	WS2812B NeoPixel, 8-LED	RGB, WS2812B protocol, GPIO 7
Motor Driver	TMC6300	3-phase, 6× PWM input, 5 V supply
BLDC Motor	Gimbal motor	7 pole pairs, low-speed high-torque

Appendix B

Additional Schematics and Source Code

B.1 User_Setup.h Configuration

The `TFT_eSPI` library requires a correctly configured `User_Setup.h` file. The settings used in this prototype are reproduced below for reference.

```
#define USER_SETUP_INFO "User_Setup"
#define GC9A01_DRIVER
#define TFT_WIDTH   240
#define TFT_HEIGHT  240
#define TFT_MOSI  5
#define TFT_SCLK 20
#define TFT_CS   21
#define TFT_DC   22
#define TFT_RST   4
#define TFT_BL   19
#define TFT_BACKLIGHT_ON HIGH
#define LOAD_GLCD
#define LOAD_FONT2
#define LOAD_FONT4
#define LOAD_FONT6
#define LOAD_FONT7
#define LOAD_FONT8
#define LOAD_GFXFF
#define SMOOTH_FONT
#define SPI_FREQUENCY      27000000
#define SPI_READ_FREQUENCY 20000000
```

Listing B.1: TFT_eSPI User_Setup.h for the GC9A01 prototype.

Appendix C

Questionnaire and Evaluation Material

C.1 Post-Session Questionnaire

(Full questionnaire to be inserted here prior to final submission.)

Appendix D

Full Firmware Source Code